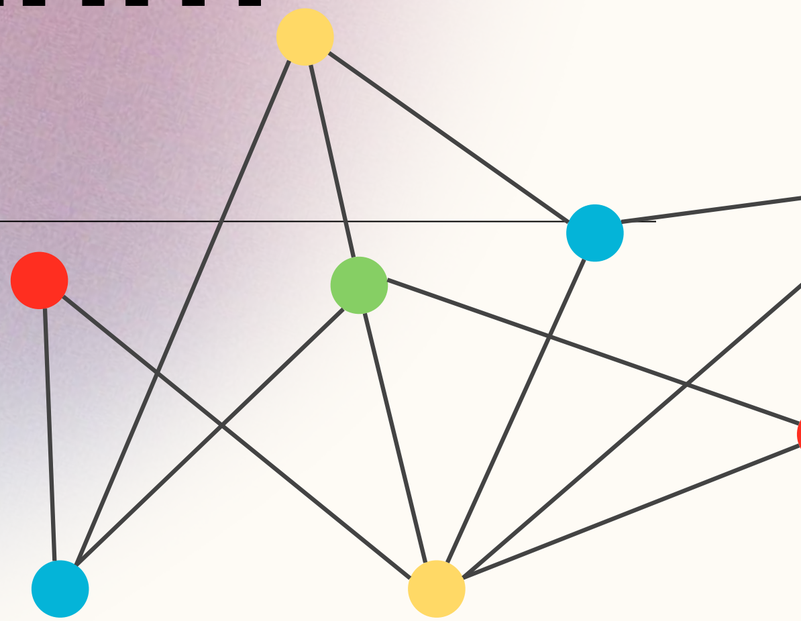
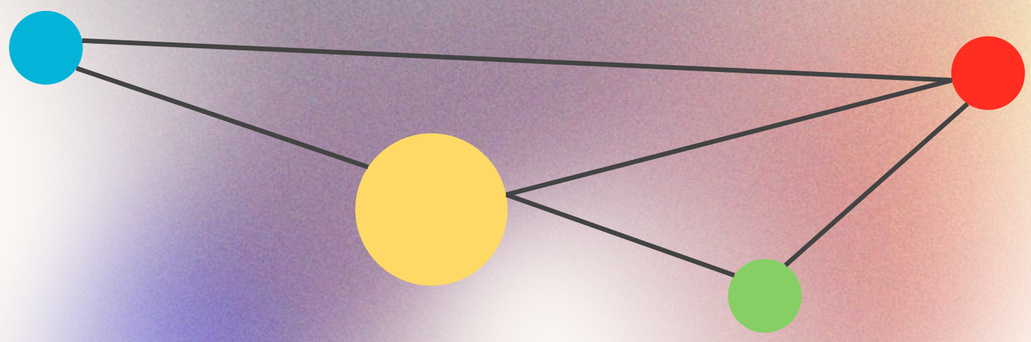
Tarjan's Strongly Connected Components Algorithm

Kyle Sullivan
Connie Deng

CS 375 Final Project



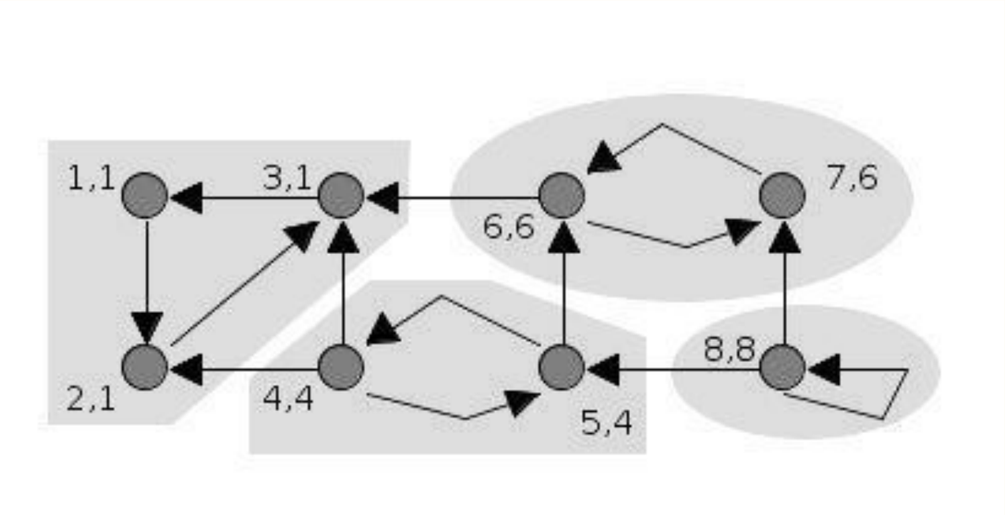
Overview



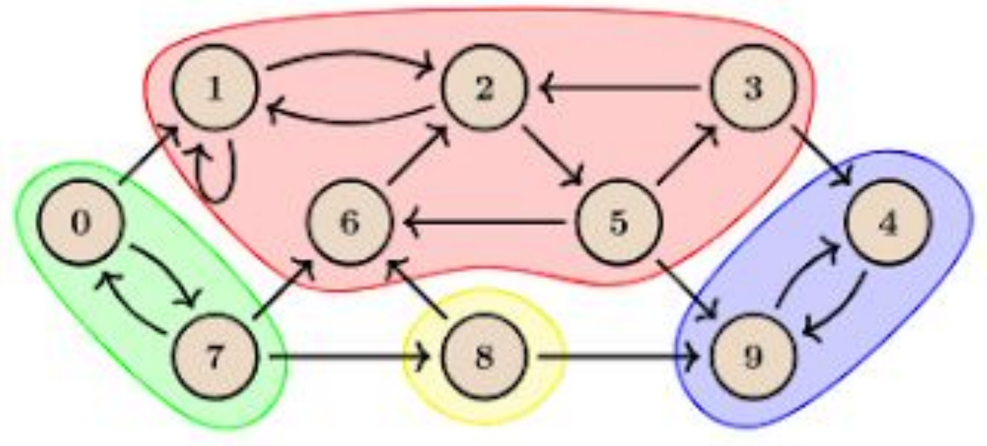
- Background
- Problem Statement
- Review of Tarjan's Algorithm
- Comparisons Between Strongly Connected Components Algorithms
- Conclusion

Introduction

I. **What is a Strongly Connected Graph?**
   A. According to Tarjan, for all vertices v & w
      1. there is a path from v to w
      2. there is a path from w to v



II. **What does Tarjan's Algorithm Aim to Accomplish / Accomplished?**
   A. Given a directed graph, Tarjan's finds strongly connected components
   B. First linear time SCC finding algorithm
   C. Requires ONE DFS pass & Memory efficient!



Problem Statement

How does Tarjan's algorithm work, and how does its performance compare to other algorithms for finding strongly connected components?

Objectives

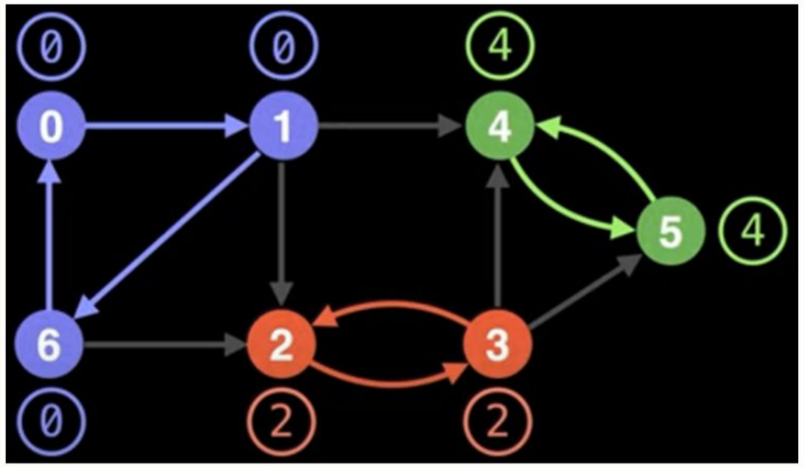
- **Understand implementation of Tarjan's Algorithm**
- **Compare Tarjan's Algorithm with other SCC finding algorithms**
- **Understand applications of Tarjan's Algorithms (in general SCC finding algorithms)**

Tarjan's Algorithm in Action

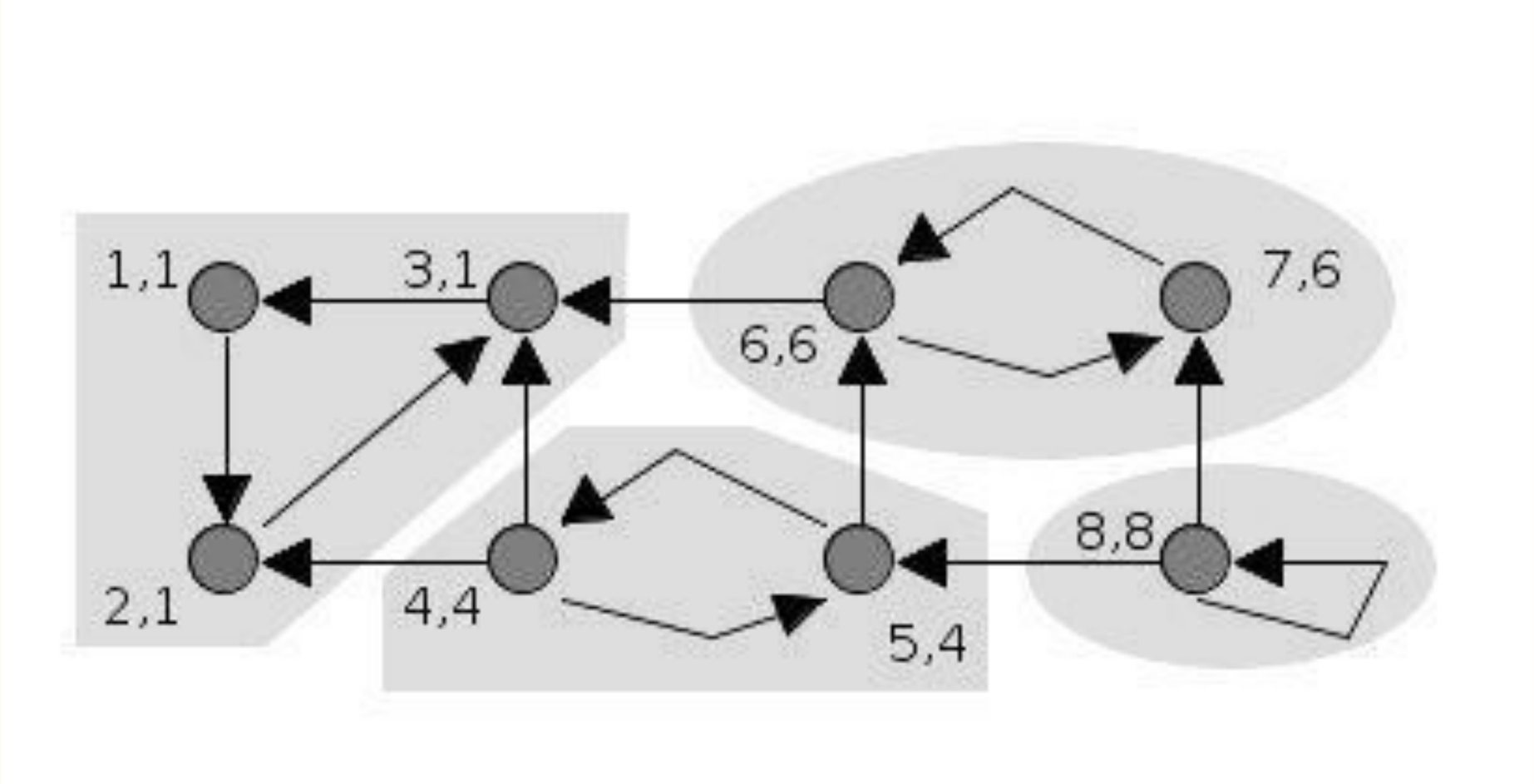
DFS, with an additional stack for tracking component candidates, and a value, LOWLINK, for the reachable vertex with the smallest discovery time

When finished searching adjacent vertices, determine if the current vertex is a component head. If so, build component

Component roots have equal LOWLINK and discovery time. All vertices with equivalent LOWLINK are in the same component



Quick Animation of Tarjan’s

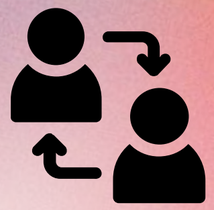


Practical Applications

GENERALLY, problems that can be reduced down to a directed graph, and deal with cyclic components, are likely to use Tarjan's Algorithm. Some examples include:

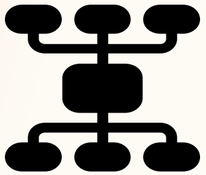
Within Operating Systems

- Deadlock detection



Social Network Analysis

- SCCs can be used here to identify strong social network communities



Dependency Resolution

- SCCs can find interdependencies between software modules

Quick Comparisons

- **Kosaraju's Algorithm:**
  - Uses two DFS passes, requires graph traversal, uses finish times
  - **Time:** O(|V| + |E|) **Space**: O(|V| + |E|)
- **Path Based Strong Component Algorithm:**
  - Similar to Tarjan's but uses an additional stack to keep track of path.
  - **Time:** O(|V| + |E|) **Space**: O(|V| + |E|)
- **Divide-and-Conquer Approach:**
  - Recursively splits graphs and computes SCCs at each partition
  - **Time**: O(|E+V| log |V|) **Space**: O(|V| + |E|)
- **Brute Force Approach:**
  - Visit every node, and do a DFS with each node as source
  - **Time**: $O(|V|^2)$ **Space:** O(|V|)

Wrap Ups and Conclusions

- ➔ Tarjan's algorithm is a DFS based approach, and finds strong connected components within a graph, and runs optimally @ O(V+E) time complexity, with a memory complexity of O(V+E).
- ➔ Inherits DFS's sequential order, meaning Tarjan does not scale well on distributed systems / large graphs systems
- ➔ Has a lot of practical applications in multiple domains!

Q&A Session

Thank you for listening!

References

- King, V. (1986). Robert (Bob) Endre Tarjan. https://amturing.acm.org/award_winners/tarjan_1092048.cfm
- Tarjan, R. (1972, June). Depth-First Search and Linear Graph Algorithms. SIAM Journal on Computing, 1(2), 146-160. https://web.archive.org/web/20170829214726/http://www.cs.ucsb.edu/~gilbert/cs240a/old/cs240aSpr2011/slides/TarjanDFS.pdf
- A. Neeraja, C. M. Begum, K. V. Reddy, R. H. Koushal and D. K. V, "An Enhanced Deadlock Detection Methodology Using Tarjan's Algorithm," 2024 8th International Conference on Computational System and Information Technology for Sustainable Solutions doi: 10.1109/CSITSS64042.2024.10816916
- Fleischer, L.K., Hendrickson, B., Pınar, A. (2000). On Identifying Strongly Connected Components in Parallel. In: Rolim, J. (eds) Parallel and Distributed Processing. IPDPS 2000. Lecture Notes in Computer Science, vol 1800. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-45591-4_68
- By LynX - Own work, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=24829107